Chapter 10

NoSQL & Choosing a Database

Why this chapter matters: Chapter 9 showed how far a relational database can go and what it costs to scale it. NoSQL systems **change the deal**: they give up some relational guarantees (joins, multi-row ACID, ad-hoc queries) in exchange for **horizontal scale, flexible schemas, very high throughput and availability** for specific access patterns. Picking the right store — or the right *combination* of stores — is one of the most visible decisions in any system design answer.

⚡ **QUICK RECALL** — **Chapter map**

- 10.1 What NoSQL is (and isn't), BASE → 10.2 Modelling for queries → 10.3 Key-value → 10.4 Document → 10.5 Wide-column → 10.6 Graph
- 10.7 Other specialised stores: search, time-series, OLAP, NewSQL, vector → 10.8 How NoSQL partitions & replicates
- 10.9 SQL vs NoSQL decision guide & polyglot persistence → 10.10 Capacity examples → 10.11 Pitfalls → 10.12 Practice, Q&A, cheat sheet

10.1 What NoSQL is — and what it isn't

NoSQL ("not only SQL") is a collection of data items represented in a **key-value store, document store, wide-column store, or graph database**. Data is typically **denormalized**, and **joins are generally done in the application code**. Most NoSQL stores lack true ACID transactions across many records and favour **eventual consistency**.

	ACID (relational)	BASE (typical NoSQL)
Stands for	Atomicity, Consistency, Isolation, Durability	**B**asically **A**vailable, **S**oft state, **E**ventual consistency
Meaning	Transactions are all-or-nothing, always valid, isolated, durable	**Basically available**: the system guarantees availability. **Soft state**: the state may change over time, even without input. **Eventual consistency**: the system becomes consistent over a period of time, given that it doesn't receive input during that period
Side of CAP	Consistency first (CP-leaning)	In comparison with the CAP theorem, **BASE chooses availability over consistency** (AP-leaning) — Chapter 5
Strength	Correctness, complex queries	Scale-out, availability, throughput, flexibility

✎ **CORRECTION TO YOUR ORIGINAL NOTE** — **Myths worth correcting**

- **"NoSQL means no schema."** There is always a schema — it moves from the database into your application code (*schema-on-read*). Use schema validation (MongoDB validators, protobuf/JSON Schema) to avoid chaos.
- **"NoSQL is always faster / scales better."** It is faster and scales better *for the access patterns it was designed for*. A key-value lookup is superb; an ad-hoc analytical query is terrible.
- **"NoSQL has no transactions."** Many now offer limited ones: MongoDB multi-document transactions, DynamoDB transactions, Cassandra lightweight transactions (Paxos-based compare-and-set), Redis MULTI/Lua. They are narrower or slower than relational ones.
- **"Relational can't scale."** It can scale a long way (Chapter 9); NoSQL mainly makes *horizontal* scaling of writes and data size a built-in feature rather than an engineering project.

10.2 Modelling for queries, not for entities

The biggest mindset change: in a relational database you **model the data** (normalised, queried flexibly later). In most NoSQL stores you **model the queries**: list the access patterns first, then shape the data so each is served by **one lookup on one partition**.

Question	Relational habit	NoSQL habit
How do I start?	Draw entities and relations, normalise to 3NF	**List the queries** (“get last 50 messages of a chat”, “get order by id”) and their frequency/latency needs
Joins?	Joined at query time by the database	**Pre-joined (denormalised)** at write time, or joined in the application
Duplication?	Avoid (single source of truth)	**Accept it**: one table/collection per query pattern; storage is cheap, cross-node joins are not
Schema change	ALTER TABLE (migration)	Add fields freely; handle versions in code; backfill lazily

Embed vs reference (document model)

Relationship	Choose	Why
One-to-few (order → 3 line items; user → 2 addresses)	**Embed** inside the parent document	One read returns everything; atomic update of the whole document
One-to-many (blog post → thousands of comments)	**Reference** (child documents with the parent id) or bucket them	Arrays that grow without bound bloat the document and hit size limits
Many-to-many (students ↔ courses)	Reference with ids on one or both sides	Avoid duplicating large, frequently-changing data
Data that changes together / read together	Embed	Locality: one disk read
Data read independently or updated at a different rate	Reference	Avoid rewriting big documents for tiny changes

★ **MUST REMEMBER**

- **Design rule for NoSQL:** know your access patterns → choose the partition/shard key that serves them → denormalise to avoid cross-partition work → plan for the data’s growth (no unbounded partitions/arrays).

10.3 Key-value stores

Abstraction: hash table. A **key-value store** generally allows for **O(1) reads and writes** and is often backed by memory or SSD. Some data stores can maintain keys in **lexicographic order**, allowing efficient retrieval of key ranges, and can allow storing of metadata with a value. They provide **high performance** and are often used for simple data models or for rapidly-changing data, such as an in-memory cache layer. Since they offer only a limited set of operations, **complexity is shifted to the application layer** if additional operations are needed. A key-value store is the basis for more complex systems such as a document store and, in some cases, a graph database.

Store	Nature	Highlights	Typical use
Redis	In-memory data-structure server (optional persistence)	Strings, hashes, lists, sets, **sorted sets**, streams, bitmaps, HyperLogLog, geo; TTL; Lua scripting; pub/sub; replication; **Cluster: 16,384 hash slots**; commands run on a single thread (atomic, no locks)	Cache, sessions, leaderboards, rate limiting, queues/streams, real-time counters
Memcached	Pure in-memory cache	Simple get/set, multi-threaded, no persistence, no replication; client-side sharding	Plain caching (Chapter 11)
DynamoDB	Managed key-value + document (AWS)	Partition key (+ sort key), single-digit-ms latency at any scale, on-demand or provisioned capacity, global secondary indexes, optional strong reads, transactions, global tables	Carts, user data, metadata, serverless back ends

Store	Nature	Highlights	Typical use
etcd / Consul KV / ZooKeeper	Small, strongly consistent KV (Raft/ZAB)	Watches, leases, compare-and-swap	Config, service discovery, leader election (Chapter 8)
RocksDB / LevelDB	Embedded LSM store (a library)	Fast local writes; used inside other databases	Building block (CockroachDB, Kafka Streams state...)

```
SET session:abc123 '{"uid":42}' EX 1800        # session with 30-minute TTL
INCR rate:user42:202610100930                  # fixed-window rate limit counter...
EXPIRE rate:user42:202610100930 60             # ...that disappears after a minute
ZADD leaderboard 4200 "alice"                  # sorted set = automatic leaderboard
ZINCRBY leaderboard 50 "alice"
ZREVRANGE leaderboard 0 9 WITHSCORES           # top 10        |  ZREVRANK leaderboard "alice" -> rank
```

◆ **SENIOR INSIGHT** — **Using Redis well (and where it bites)**

- **Persistence**: RDB snapshots (periodic, fast restart, can lose recent writes) and AOF (append-only log; `everysec` loses ≤ ~1 s; `always` is safest and slowest). Treat Redis as a **cache or ephemeral store** unless you have designed its durability (Chapter 2: fsync trade-offs).
- **Memory is the cost driver**: plan ~1.5–2× raw data size for keys, pointers and fragmentation; set `maxmemory` and an eviction policy (`allkeys-lru`...). Beware **big keys** and slow O(n) commands (KEYS *, huge SMEMBERS) — they block the single execution thread.
- **Replication is asynchronous** (some acknowledged writes can be lost on failover); Redis Cluster splits keys over hash slots, so multi-key operations need keys in the same slot (hash tags). Redis is not a safe distributed lock by itself under partitions — use fencing tokens or a CP system for critical locks.
- Limits of KV: **no ad-hoc queries**, only lookup by key (or range, if ordered). If you find yourself scanning, you picked the wrong model.

10.4 Document stores

Abstraction: key-value store with documents stored as values. A **document store** is centred around documents (XML, JSON, binary, etc.), where **a document stores all information for a given object**. Document stores provide APIs or a query language to query based on the **internal structure of the document itself**. Many key-value stores include features for working with a value's metadata, blurring the lines between these two storage types.

```
{ "_id": "order_9981", "userId": 42, "status": "PAID",
  "items":    [ {"sku": "PEN-1", "qty": 2, "price": 10}, {"sku": "BK-7", "qty": 1, "price": 250} ],
  "shipping": { "city": "Delhi", "pin": "110001" },
  "createdAt": "2026-10-10T09:30:00Z" }          // everything about the order in ONE document

db.orders.createIndex({ userId: 1, createdAt: -1 })
db.orders.find({ userId: 42, status: "PAID" }).sort({ createdAt: -1 }).limit(20)
```

- **Organisation**: based on the underlying implementation, documents are organised by **collections, tags, metadata or directories**. Although documents can be organised or grouped together, **documents may have fields that are completely different from each other**.
- **Examples**: **MongoDB** and **CouchDB** also provide a SQL-like language to perform complex queries (MongoDB: query language + aggregation pipeline); **DynamoDB** supports both key-values and documents; Firestore, Couchbase, Elasticsearch (as a search/document store).
- **Strengths**: high flexibility; often used for working with **occasionally changing data**; one read returns the whole aggregate (no joins); secondary indexes on any field; natural mapping to application objects/JSON.
- **Scaling**: replica sets (one primary, several secondaries, automatic election) for HA and read scaling; **sharding by a shard key** (hashed or ranged) for writes and size — the shard key choice matters as in Chapter 9.

- **Consistency**: single-document operations are atomic; multi-document transactions exist but cost performance and are best avoided in the hot path — model so that one document is the unit of consistency.

> ✖ COMMON MISTAKE — **Document-store pitfalls**
> • **Unbounded arrays** (all comments inside the post) → huge documents (MongoDB's limit is 16 MB) and rewrites; bucket or reference instead. **Monotonic shard keys** (timestamps, ObjectId) → every insert hits one shard (hot shard). **Overusing** `$lookup` (joins) means you modelled it relationally. **Schema drift** without validation → every field ends up in five spellings.

10.5 Wide-column stores

Abstraction: nested map `ColumnFamily<RowKey, Columns<ColKey, Value, Timestamp>>`. A **wide-column store**'s basic unit of data is a **column** (name/value pair). A column can be grouped in **column families** (analogous to a SQL table); **super column families** further group column families. You can access each column independently with a **row key**, and columns with the same row key form a row. **Each value contains a timestamp** for versioning and for conflict resolution.

Google introduced **Bigtable** as the first wide-column store, which influenced the open-source **HBase** (often used in the Hadoop ecosystem) and **Cassandra** (born at Facebook; also ScyllaDB). Stores such as Bigtable, HBase and Cassandra **maintain keys in lexicographic order**, allowing efficient retrieval of selective key ranges. They offer **high availability and high scalability** and are often used for **very large data sets**.

Modelling example (Cassandra CQL): chat messages, always read as "newest N of one chat"

```
CREATE TABLE messages_by_chat (
  chat_id   uuid,
  sent_at   timeuuid,
  sender_id bigint,
  body      text,
  PRIMARY KEY ((chat_id), sent_at)            -- (partition key), clustering column
) WITH CLUSTERING ORDER BY (sent_at DESC);

SELECT * FROM messages_by_chat WHERE chat_id = ? LIMIT 50;   -- ONE partition, already sorted
-- Partition grows forever?  Bucket it:  PRIMARY KEY ((chat_id, month), sent_at)
```

Why writes are so fast: the LSM-tree write path

```
  1) write ---> [ commit log ]  append-only, sequential (durability, like a WAL)
  2) write ---> [ memtable ]    sorted, in RAM  --> the write is acknowledged here
  3) memtable full ---> FLUSH ---> [ SSTable 1 ] [ SSTable 2 ] ...   (immutable sorted files on disk)
  4) background COMPACTION merges SSTables, drops overwritten data and tombstones

  read = check memtable + bloom filters -> read the few SSTables that may hold the key -> merge results
```

Concept	Meaning
Partition key	Hashed (consistent hashing ring) to decide which nodes store the row; all rows of a partition live together
Clustering columns	Sort order of rows *inside* a partition → fast range slices (time-ordered data)
Tunable consistency	Per query: ONE, QUORUM, ALL, LOCAL_QUORUM; R + W > N for read-your-writes (Chapter 5)
Replication factor (RF)	Number of copies (commonly 3), placed across racks/AZs/data centres
Tombstones	Deletes are writes of a marker; reading many tombstones is slow → avoid queue-like delete patterns
No joins, no ad-hoc filters	Queries must match the primary key design; one denormalised table per query

◆ **SENIOR INSIGHT** — **When wide-column shines — and anti-patterns**

- **Great for**: **time series**, IoT/telemetry, messaging history, activity feeds, audit logs, user-event stores — **write-heavy, huge volume, key-based access, always-on multi-region**.
- **Anti-patterns**: unbounded/giant partitions (hundreds of MB) and **hot partitions** (one celebrity key); secondary indexes on high-cardinality columns; read-before-write; frequent deletes (tombstones); needing joins/aggregations/ad-hoc queries.
- Trade-off you can state: **very high write throughput and availability (AP, PA/EL)** in exchange for modelling discipline and weak multi-row transactions.

10.6 Graph databases

Abstraction: graph. In a **graph database**, **each node is a record and each arc is a relationship between two nodes**. Graph databases are optimised to represent **complex relationships with many foreign keys or many-to-many relationships**. They offer **high performance for data models with complex relationships, such as a social network**.

```
(Alice:User) --FOLLOWS--> (Bob:User) --FOLLOWS--> (Carol:User)
(Alice:User) --MEMBER_OF--> (Chess:Group) <--MEMBER_OF-- (Bob:User)
Nodes have labels + properties; relationships have a type, a direction and properties.

// Cypher (Neo4j): friends-of-friends I don't follow yet, ranked by mutual connections
MATCH (me:User {id: 1})-[:FOLLOWS]->(:User)-[:FOLLOWS]->(rec:User)
WHERE NOT (me)-[:FOLLOWS]->(rec) AND rec <> me
RETURN rec.name, count(*) AS mutual ORDER BY mutual DESC LIMIT 10;
```

- **Why they win**: traversals follow **direct pointers between neighbouring nodes** (“index-free adjacency”), so the cost depends on the part of the graph visited, not on total data size. The SQL equivalent of friends-of-friends-of-friends is a chain of self-joins or recursive CTEs that gets slower with every hop.
- **Use cases**: social networks and friend recommendation, **fraud detection** (rings of accounts sharing devices/cards), knowledge graphs, identity/permission graphs, network/IT topology, route and dependency analysis, supply chains.
- **Examples**: Neo4j, Amazon Neptune, JanusGraph, TigerGraph; Twitter’s **FlockDB** was a simple distributed graph store for follower relationships (it has since been retired).
- **Limits**: densely connected data is **hard to shard** (cutting the graph cuts edges), fewer tools and skills than SQL, whole-graph analytics are better done in batch systems, and some engines are accessible only through REST/bolt APIs. Many teams model simple graphs (followers) in SQL or a wide-column store and add a graph DB only for deep traversals.

✎ **CORRECTION TO YOUR ORIGINAL NOTE** — **“Graph databases are relatively new and not widely used”**

- That reflects older material. Graph databases are now an established specialised category (fraud, recommendations, knowledge graphs), though still niche compared with relational and key-value stores — so choose one when relationship depth is the core query, not by default.

10.7 Other specialised stores

Category	Examples	Core idea	Use for	Caveats
Search engine	Elasticsearch, OpenSearch, Solr	**Inverted index** (term → documents), relevance scoring (BM25), analyzers, aggregations, near-real-time	Full-text search, autocomplete, log analytics, faceted filters	**Not a source of truth**: eventual consistency, no real transactions; feed it from the primary DB via CDC/outbox (Chapter 8)
Time-series	InfluxDB, TimescaleDB, Prometheus, VictoriaMetrics	Append-mostly data keyed by time; compression, downsampling, retention policies, time-bucket aggregates	Metrics, IoT sensors, monitoring, finance ticks	High-cardinality labels explode memory; plan retention/rollups
Columnar / OLAP warehouse	BigQuery, Snowflake, Redshift, ClickHouse	**Store by column**: read only needed columns, compress heavily, vectorised scans over billions of rows	Analytics, BI, reporting	Slow for single-row lookups and frequent updates — separate from OLTP
NewSQL	Google Spanner, CockroachDB, TiDB, YugabyteDB	**SQL + ACID + horizontal scale** via automatic sharding and Raft consensus per range	Globally distributed, strongly consistent OLTP (money, inventory) beyond a single node	Higher write latency (consensus, cross-region), operational and cost complexity
Vector database	pgvector, Pinecone, Milvus, Weaviate	Store embeddings; **approximate nearest-neighbour** (HNSW/IVF) similarity search	Semantic search, recommendations, retrieval for AI apps	Approximate results; combine with filters; keep source data elsewhere
Object / blob	S3, GCS, Azure Blob	Flat namespace of immutable objects (Chapter 2)	Images, video, backups, data lakes	No queries beyond key/prefix; high latency per request

OLTP vs OLAP — do not mix them

	OLTP (transactions)	OLAP (analytics)
Typical query	Fetch/update one order or user — milliseconds	Sum revenue by region for 3 years — seconds–minutes
Pattern	Many small reads/writes, row-oriented, indexed	Few huge scans and aggregations, column-oriented
Store	PostgreSQL, MySQL, DynamoDB, MongoDB…	BigQuery, Snowflake, ClickHouse, Redshift
Data flow	Source of truth	Fed from OLTP by ETL/ELT or CDC; heavy reports never run on the primary

10.8 How NoSQL stores partition and replicate

Most NoSQL stores are **shared-nothing, scale-out** systems (Chapter 5). Two decisions define them: how data is **partitioned** and how it is **replicated**.

Store	Partitioning	Replication & consistency (default)	CAP / PACELC tendency
Redis Cluster	16,384 hash slots mapped to masters	Async master → replicas; failover by election; can lose acked writes	AP-ish for writes, tunable with WAIT
DynamoDB	Hash of partition key (adaptive splitting)	3 AZ replicas; eventually-consistent reads default, strongly consistent read optional (leader)	PA/EL by default; strong option

Store	Partitioning	Replication & consistency (default)	CAP / PACELC tendency
Cassandra / ScyllaDB	Consistent-hash ring with virtual nodes; partition key	**Leaderless**, RF copies, quorum tunable; hinted handoff, read repair, anti-entropy	PA/EL (AP), tunable toward C
MongoDB	Hashed or ranged shard key; chunks balanced across shards	Replica set: primary + secondaries, election; write/read concerns (majority)	CP-leaning with majority concerns
HBase / Bigtable	Range-partitioned regions/tablets, sorted by row key	Single owner per region; data in a distributed FS (HDFS/Colossus)	PC/EC: strongly consistent per row
Spanner / CockroachDB	Ranges, each a Raft group	Majority consensus per range; global transactions	PC/EC (CP): consistency over latency

- **Consistent hashing** (Chapter 14): nodes and keys are placed on a ring; adding a node moves only ~1/N of the keys; **virtual nodes** even out load and speed up rebalancing.
- **Range vs hash partitioning** (as in Chapter 9): hash = even load, poor range scans; range = efficient scans, hot-spot risk with sequential keys (timestamps).
- **Hot partitions** are the #1 operational issue. Remedies: better/composite keys, **key salting** (append a random suffix 0–N and fan-out reads), write buffering, caching in front, splitting hot ranges.
- **Quorum math** (Chapter 5): with RF = 3, QUORUM = 2; reads and writes at QUORUM overlap (2 + 2 > 3) and survive one node down.
- **Rebalancing** is automatic but costs I/O and network — throttle it; add capacity **before** disks hit ~70% (compaction needs free space).

10.9 SQL vs NoSQL — the decision guide

Reasons for SQL	Reasons for NoSQL
Structured data **Strict schema** **Relational data** **Need for complex joins** **Transactions** **Clear patterns for scaling** **More established**: developers, community, code, tools, etc. **Lookups by index are very fast**	**Semi-structured data** **Dynamic or flexible schema** **Non-relational data** **No need for complex joins** **Store many TB (or PB) of data** **Very data-intensive workload** **Very high throughput for IOPS**

Sample data well-suited for NoSQL:

- **Rapid ingest of clickstream and log data** (wide-column / time-series / Kafka → lake).
- **Leaderboard or scoring data** (Redis sorted sets).
- **Temporary data, such as a shopping cart** (key-value with TTL; AP carts).
- **Frequently accessed ('hot') tables** (cache in front of the primary store).
- **Metadata / lookup tables** (key-value, DynamoDB).

Use-case → store

Requirement	Good fit	Why
Orders, payments, inventory, anything needing multi-row ACID and joins	**PostgreSQL/MySQL** (or NewSQL if it must scale out)	Transactions, constraints, flexible queries
Sessions, carts, counters, leaderboards, rate limits, caching	**Redis** / DynamoDB	Sub-ms access, TTL, rich structures
Product catalogue with varied attributes, content, user profiles	**MongoDB / document** (or PostgreSQL JSONB)	Flexible schema; one read per aggregate
Messaging history, activity feeds, IoT/time-series at huge scale	**Cassandra / Scylla / Bigtable**	Linear write scaling, multi-region, partition-ordered reads
Friends-of-friends, fraud rings, recommendations, knowledge graphs	**Graph DB**	Deep traversals
Full-text search, autocomplete, log search	**Elasticsearch/OpenSearch**	Inverted index, relevance
Dashboards, reports over billions of rows	**Columnar warehouse**	Scan/aggregate speed
Images, video, backups	**Object storage + CDN**	Cheap, durable, scalable

◆ **SENIOR INSIGHT** — **Polyglot persistence — and its price**

- Real systems use **several** stores, each for what it does best. Example e-commerce: PostgreSQL (orders/payments) • Redis (carts, sessions, cache) • Elasticsearch (product search) • Cassandra or DynamoDB (event history) • S3 + CDN (images) • a warehouse (analytics).
- **The price**: more systems to run, secure, back up and monitor; and **data now lives in many places**. Keep one **system of record** per fact and propagate changes to the others through events/CDC/outbox (Chapter 8) — the others are **derived, rebuildable views**.
- **Default advice**: start with a relational database + cache (PostgreSQL covers JSONB, full-text, geo, queues for small scale). Add a specialised store only when a concrete access pattern or scale limit demands it (the Chapter 9 ladder). “Boring technology” is a feature.

10.10 Capacity examples

Example A — Cassandra for chat/feed history (100 M messages/day, 1 KB each, RF = 3, nodes with 4 TB disks kept ≤ 50% full for compaction headroom)

Step	Calculation	Result
Write rate	100 M ÷ 86,400 ≈ 1,160/s average; peak ×3	≈ **3,500 writes/s** — trivial for a small cluster
Raw data / day	100 M × 1 KB	100 GB/day
With replication ×3	100 GB × 3	300 GB/day ≈ **110 TB/year**
Usable per node	4 TB × 50%	2 TB
Nodes for year 1	110 TB ÷ 2 TB	≈ **55 nodes** (plus TTL/archiving would cut this)
Insight		**Storage, not QPS, sizes the cluster.** Consider TTL, compression and tiering to cold storage

Example B — DynamoDB provisioned capacity (*check current docs; limits and pricing change*)

1 write capacity unit (WCU) = one write/s of up to 1 KB. 1 read capacity unit (RCU) = one strongly consistent read/s of up to 4 KB, or two eventually consistent reads/s.

```
Writes : 5,000 writes/s x 2 KB items   -> 5,000 x ceil(2/1) = 10,000 WCU
Reads  : 20,000 reads/s x 3 KB, eventually consistent -> each read = 1 unit of 4 KB; eventual = 0.5 RCU
         20,000 x 0.5 = 10,000 RCU        (strongly consistent would need 20,000 RCU)
Hot key: capacity is split across partitions - one very hot partition key can throttle
         even when the table as a whole has spare capacity.
```

Example C — Redis sessions (10 M active sessions × 2 KB)

Raw ≈ 20 GB; with ~1.5× overhead ≈ **30 GB**. A 3-primary Redis Cluster with ~16 GB per primary leaves comfortable headroom (+ one replica each for failover). Add TTLs so abandoned sessions expire, and keep `maxmemory` below physical RAM for the fork/snapshot overhead.

10.11 Pitfalls & anti-patterns

Anti-pattern	Why it hurts	Better
Choosing NoSQL because it's "web scale"	You lose joins/transactions without needing the scale	Measure first; stay relational until a clear limit is reached
Modelling like a relational schema	Joins in app code → N+1 network calls, slow	Model per query; denormalise; embed where read together
Hot partition / monotonic key	One node takes all load; throttling	Hash/composite/salted keys; spread writes
Unbounded growth (arrays, partitions, collections)	Giant rows, GC/compaction pain, size limits	Bucket by time; TTL; archive
Assuming strong consistency	Stale reads, lost updates, duplicates	Know the defaults; use quorum/strong reads or CAS where needed; idempotency
Redis/memory store as the only copy	Data loss on restart/failover	Use for cache/ephemeral, or design persistence + replication + backups
No schema validation / versioning	Five formats for the same field	Validators, schema registry, version field
Search engine as source of truth	Index rebuilds lose data; no transactions	Keep primary DB; reindex from it
Mixing OLTP and analytics on one store	Reports slow production	Replicate to a warehouse
Ignoring operations	Backups, upgrades, monitoring, capacity (compaction headroom)	Managed service or dedicated expertise

10.12 Practice questions (try before looking)

P1. Design the data layer for a chat app with 50 M DAU, messages read mostly as 'latest 50 of a conversation'. Which store, and what is the key?

Answer: Wide-column (Cassandra/Scylla) or DynamoDB: partition key = conversation id (+ time bucket), clustering/sort key = timestamp DESC; reads are one partition. Presence/typing in Redis; media in object storage.

P2. A product catalogue has thousands of product types with different attributes, read heavily by product id and filtered by category. Store?

Answer: Document store (MongoDB) or PostgreSQL JSONB; search via Elasticsearch for text/facets; cache hot products in Redis.

P3. Your Cassandra table partitions by `country` and one partition is 40 GB. What's wrong and the fix?

Answer: Low-cardinality, skewed key → giant hot partition. Use a high-cardinality key (user id) or add buckets (country, day/hash) so partitions stay small and spread out.

P4. You need 'top 100 players' and each player's rank with millions of score updates per minute. Store?

Answer: Redis sorted sets (ZADD/ZINCRBY, ZREVRANGE, ZREVRANK), sharded by game/region; persist final scores to a durable DB.

P5. DynamoDB: 8,000 writes/s of 1.5 KB items. WCUs required?

Answer: Each write rounds up to 2 KB = 2 WCU → 8,000 × 2 = **16,000 WCU** (assuming provisioned mode and even key distribution).

P6. Fraud team wants to find accounts sharing devices/cards/addresses within 4 hops. Store?

Answer: Graph database (traversals) fed from the transactional DB via CDC; results flow back as risk flags.

Interview Q&A (30-second answers)

Q1. What are the NoSQL families and an example use for each?

Key-value (Redis/DynamoDB: sessions, caches, counters), document (MongoDB: catalogues, profiles), wide-column (Cassandra/HBase: time-series, messaging), graph (Neo4j: relationships, fraud).

Q2. ACID vs BASE?

ACID prioritises correctness via transactions; BASE (basically available, soft state, eventual consistency) prioritises availability and scale, accepting temporary inconsistency.

Q3. How do you model data in NoSQL differently?

Start from access patterns, denormalise, one table/collection per query, embed data read together, reference unbounded or independently changing data, choose partition keys for even spread.

Q4. Why is Cassandra good at writes?

LSM design: append to a commit log + in-memory memtable, flush sorted immutable SSTables, compact in the background — sequential I/O, no read-before-write, leaderless replication.

Q5. What is a hot partition and how do you fix it?

A key/partition receiving disproportionate traffic. Use higher-cardinality or composite keys, salting with fan-out reads, caching, splitting/bucketing, adaptive capacity.

Q6. When would you choose a graph database?

When queries are deep relationship traversals (friends-of-friends, fraud rings, recommendations) that would need many self-joins in SQL.

Q7. Why isn't Elasticsearch a primary database?

Eventually consistent, no transactions, mappings/reindexing pain; use it as a derived index fed from the system of record.

Q8. SQL or NoSQL for a new product?

Default to relational + cache; adopt NoSQL when the access pattern (key-value, huge write volume, flexible documents, graph) or scale clearly requires it, and keep one source of truth.

Q9. What is polyglot persistence and its risk?

Using multiple specialised stores in one system. Risk: operational overhead and data synchronisation; mitigate with one system of record and event-driven propagation.

Q10. What is NewSQL?

Distributed SQL databases (Spanner, CockroachDB, TiDB) offering ACID transactions and SQL with automatic sharding via consensus — at the price of higher write latency and cost.

Chapter 10 Cheat Sheet

⚡ **QUICK RECALL** — **Revise this in 2 minutes**

- **NoSQL** = key-value, document, wide-column, graph. Denormalised, joins in the app, mostly eventual consistency. **BASE** = basically available, soft state, eventual consistency (availability over consistency).
- **Model the queries, not the entities:** list access patterns → partition key → denormalise; embed one-to-few, reference one-to-many; avoid unbounded growth.
- **Key-value** (hash table, O(1)): Redis (structures, TTL, 16,384 slots), Memcached (cache only), DynamoDB (partition + sort key). Cache, sessions, counters, leaderboards.
- **Document** (key → JSON doc): MongoDB, CouchDB, DynamoDB. Flexible schema; atomic per document; beware unbounded arrays and monotonic shard keys.
- **Wide-column** (nested map ColumnFamily<RowKey, Columns<ColKey, Value, Timestamp>>): Bigtable, HBase, Cassandra. LSM write path, tunable consistency, huge write scale; query-first tables.
- **Graph** (nodes + relationships): Neo4j, Neptune. Deep traversals, fraud, recommendations; hard to shard.
- **Specialised:** search (inverted index; derived), time-series, columnar OLAP (separate from OLTP), NewSQL (SQL + ACID + scale-out, costlier writes), vector (similarity), object storage.
- **Scale:** consistent hashing / hash slots / ranges; replication factor 3; quorum R+W>N; fix hot partitions with key design and salting.
- **SQL reasons:** structured, strict schema, joins, transactions. **NoSQL reasons:** semi-structured, flexible, no joins, TB–PB, very high throughput.
- **Default:** relational + cache first; polyglot only when needed; one system of record; others derived via CDC/events.
- **Numbers:** storage often sizes NoSQL clusters (e.g., 110 TB/yr → ~55 nodes at 2 TB usable); DynamoDB 1 WCU = 1 KB write/s, 1 RCU = 4 KB strong read/s.

AMJESH https://singhamjesh.github.io/